
Integrasi Firmware ESP32 dan Library Python untuk Pengendalian Aktuator pada Kit iMCLab

Muhammad Wifaqul Azmi*¹,

Universitas Pembangunan Nasional Veteran Jawa Timur; Jalan Raya Rungkut Madya No. 1,
Gunung Anyar, Surabaya, Jawa Timur, Indonesia
Informatika, Fakultas Ilmu Komputer, Jawa Timur
e-mail: *¹23081010246@student.upnjatim.ac.id

Abstrak

Penelitian ini mengimplementasikan integrasi firmware ESP32 dan library Python untuk pengendalian aktuator pada kit iMCLab melalui komunikasi serial USB. Sistem dibangun menggunakan tiga komponen utama, yaitu firmware pada ESP32 sebagai penerima perintah, library Python imclab.py sebagai antarmuka kontrol, serta skrip pengujian untuk menjalankan skenario eksperimen secara otomatis. Pengujian dilakukan pada dua aktuator, yakni LED dan motor DC, dengan perintah berbasis teks LED dan OP dalam rentang 0–100 serta perintah X untuk menghentikan motor. Hasil pengujian menunjukkan komunikasi serial berjalan stabil, LED dapat dikendalikan sesuai nilai perintah, dan motor DC dapat berputar dengan variasi kecepatan berdasarkan nilai OP. Implementasi ini membuktikan bahwa kombinasi firmware mikrokontroler dan kontrol berbasis Python dapat digunakan sebagai metode praktis untuk otomasi pengujian aktuator pada kit praktikum.

Kata kunci—iMCLab; komunikasi serial; Python; PWM; motor DC; LED

Abstract

This study implements the integration of ESP32 firmware and a Python library to control actuators on the iMCLab kit via USB serial communication. The system consists of three main components: ESP32 firmware as a command receiver, a Python library (imclab.py) as a control interface, and testing scripts to execute experimental scenarios automatically. The evaluation focuses on two actuators, namely an LED and a DC motor, using text-based commands LED and OP within a 0–100 range, as well as the X command to stop the motor. The results indicate that serial communication is stable, the LED responds correctly to command values, and the DC motor speed varies according to the OP input. This implementation demonstrates that combining microcontroller firmware with Python-based control provides a practical approach for actuator automation and testing in educational lab kits.

Keywords—iMCLab; komunikasi serial; Python; PWM; motor DC; LED

1. PENDAHULUAN

Kit iMCLab (Internet-Based Motor Control Lab) menyediakan platform praktikum yang menggabungkan mikrokontroler, aktuator, dan antarmuka pengujian sehingga cocok untuk latihan pengendalian perangkat secara langsung [1]. Pada tugas ini, fokus implementasi adalah pengendalian dua aktuator dasar LED dan motor DC agar alur kerja bisa dipahami oleh pemula namun tetap menunjukkan konsep sistem kendali berbasis mikrokontroler. Mikrokontroler yang digunakan adalah ESP32 karena umum dipakai pada aplikasi embedded/IoT dan memiliki kemampuan komputasi serta antarmuka komunikasi yang memadai untuk eksperimen

pengendalian [2]. Dengan memilih studi kasus LED dan motor, pengujian dapat dilakukan secara bertahap mulai dari koneksi, eksekusi perintah, hingga validasi respons sistem.

Integrasi yang dibuat memerlukan jalur komunikasi yang stabil antara komputer dan ESP32, dan pendekatan yang paling sederhana untuk praktikum adalah komunikasi serial melalui USB. Pada lingkungan Arduino, komunikasi ini difasilitasi oleh kelas Serial sehingga ESP32 dapat menerima perintah dan mengirim balasan dalam bentuk teks yang mudah dibaca [3]. Di sisi komputer, bahasa Python dipilih karena praktis untuk otomatisasi pengujian dan dapat berperan sebagai “host controller” yang mengirim perintah secara berulang sesuai skenario uji [4]. Dengan demikian, proses pengendalian tidak lagi manual dari Serial Monitor, tetapi dapat dijalankan sebagai skrip yang repeatable.

Agar penggunaan lebih rapi, dibuat library Python (`imclab.py`) yang membungkus pengiriman perintah serial menjadi fungsi-fungsi sederhana seperti menyalakan LED atau mengatur persentase daya motor. Library ini juga dapat dibantu dengan proses enumerasi port agar pengguna tidak perlu menebak COM secara manual ketika perangkat sudah terdeteksi [5]. Di sisi firmware, salah satu strategi yang umum dan mudah dipahami pemula adalah membaca input serial per-baris hingga karakter penanda akhir (newline) sebelum diproses sebagai command [6]. Pola ini meminimalkan kesalahan parsing karena tiap perintah diperlakukan sebagai satu paket.

Supaya protokolnya konsisten, perintah dikirim dalam format teks “COMMAND nilai” dan diakhiri karakter akhir baris sehingga firmware tahu kapan sebuah perintah selesai dikirim. Pada Arduino, fungsi `Serial.println()` secara default menambahkan karakter carriage return dan newline setelah data, sehingga format baris-per-baris mudah dibentuk dari sisi pengirim maupun penerima [7]. Setelah parsing berhasil, firmware mengeksekusi perintah untuk mengubah kondisi aktuator, lalu mengirim balasan (acknowledgement) agar sisi Python bisa memastikan perintah benar-benar diterima. Mekanisme balasan ini penting untuk debugging karena pengguna bisa membedakan masalah komunikasi dan masalah perangkat keras.

Pengaturan kecerahan LED dan kecepatan motor DC dilakukan menggunakan PWM, dan pada ESP32 fitur PWM dapat dibangkitkan melalui perifer LEDC yang mendukung banyak kanal dan cocok untuk kontrol intensitas/kecepatan [8]. Pada praktiknya, duty cycle PWM dapat dinaikkan atau diturunkan dari skrip Python untuk melihat pengaruhnya terhadap respons aktuator, misalnya LED semakin terang atau motor semakin cepat. Referensi penggunaan PWM ESP32 di Arduino IDE juga banyak tersedia dan membantu pemula memahami konsep frekuensi, resolusi, dan duty cycle secara cepat [9]. Selama proses pengujian, pemilihan “line ending” pada Serial Monitor dan penggunaan end-marker menjadi bagian penting agar perintah yang dikirim tidak terpotong dan dapat diproses dengan benar [10].

2. METODE PENELITIAN

Penelitian ini menggunakan pendekatan implementatif dengan merancang firmware pada ESP32 serta library Python sebagai antarmuka pengendalian aktuator pada kit iMCLab. Tahap awal meliputi persiapan perangkat (kit iMCLab, kabel USB) dan perangkat lunak (Arduino IDE serta Python di komputer) untuk memastikan proses upload dan komunikasi serial dapat dilakukan. Firmware dirancang untuk menerima perintah berbasis teks dan mengeksekusi aksi pada aktuator sesuai parameter yang diberikan. Pengaturan PWM untuk LED dan motor dilakukan menggunakan perifer LEDC pada ESP32 melalui konfigurasi kanal dan penulisan duty cycle [11].

Pada sisi firmware, dibuat protokol perintah sederhana dengan format “COMMAND nilai” dan diakhiri newline agar parsing mudah dan konsisten saat pengujian. Perintah yang digunakan meliputi LED untuk kontrol intensitas LED, OP untuk mengatur kecepatan motor (persentase PWM), X untuk menghentikan motor, serta VER untuk identifikasi versi firmware. Setelah perintah dieksekusi, firmware mengirim balasan (acknowledgement) agar status eksekusi dapat dipantau dari sisi komputer. Pada sisi Python, objek koneksi serial dibangun

menggunakan `serial.Serial(...)` untuk mengirim perintah dan membaca balasan dari mikrokontroler [12].

Library `imclab.py` disusun sebagai pembungkus (wrapper) agar pengendalian cukup dilakukan melalui pemanggilan fungsi seperti `LED()` dan `op()` tanpa menulis format perintah manual. Struktur folder proyek disusun sehingga `imclab.py` berada dalam direktori yang sama dengan skrip pengujian (misalnya `tes_led.py` dan `tes_motor.py`) agar modul dapat diimpor dengan mudah. Proses pengujian dilakukan secara bertahap: uji koneksi dan respon awal, uji LED (ON/OFF dan variasi intensitas), lalu uji motor dengan variasi nilai OP. Port komunikasi dipilih otomatis dengan memindai daftar port yang tersedia menggunakan `serial.tools.list_ports.comports()` sehingga pengguna tidak perlu menebak COM secara manual [13].

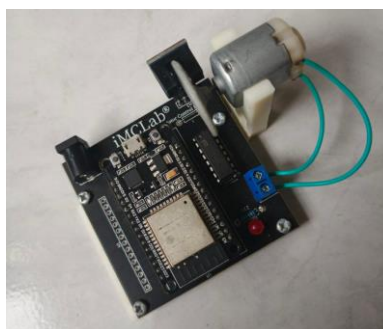
Pengujian LED dilakukan dengan memberikan nilai 0-100 untuk mengamati perubahan kondisi nyala dan memverifikasi balasan firmware pada terminal Python. Pengujian motor dilakukan dengan menaikkan nilai OP secara bertahap untuk mengamati perubahan kecepatan putar, kemudian menjalankan motor pada nilai tertentu dalam durasi lebih lama untuk observasi stabilitas. Selama pengujian, Serial Monitor Arduino ditutup ketika menjalankan skrip Python agar port tidak terkunci oleh aplikasi lain. Agar pembacaan balasan tidak menggantung tanpa batas, koneksi serial diset memiliki timeout sehingga `readline()` berhenti menunggu ketika tidak ada karakter akhir baris yang diterima [14].

Proses upload firmware dilakukan melalui Arduino IDE dengan memilih board ESP32 yang sesuai dan port COM yang terdeteksi pada sistem operasi. Jika muncul kegagalan koneksi saat flashing (misalnya kondisi boot mode tidak sesuai), tindakan perbaikan dilakukan sebelum pengujian dilanjutkan agar firmware benar-benar berjalan pada perangkat. Setelah upload berhasil, validasi awal dilakukan dengan mengirim perintah melalui Serial Monitor untuk memastikan firmware merespons, lalu pengujian dipindahkan ke skrip Python untuk menjalankan eksperimen secara repeatable. Jika proses upload firmware gagal karena perangkat tidak masuk download mode, penanganan dilakukan dengan mengikuti panduan troubleshooting esptool, misalnya masuk bootloader secara manual saat proses koneksi [15].

3. HASIL DAN PEMBAHASAN

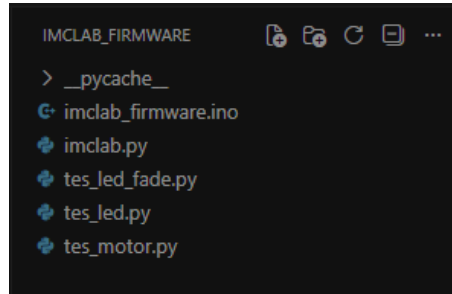
3.1 Hasil Implementasi Sistem

Pada tahap ini, sistem pengendalian aktuator pada kit iMCLab berhasil diimplementasikan dengan memanfaatkan komunikasi serial antara komputer dan ESP32. Implementasi terdiri dari tiga komponen utama, yaitu firmware pada ESP32 yang menerima perintah berbasis teks, library Python `imclab.py` sebagai antarmuka pemanggilan fungsi, serta skrip pengujian (misalnya `tes_led.py` dan `tes_motor.py`) untuk menjalankan skenario kontrol secara otomatis. Alur kerja sistem dimulai ketika skrip Python mengirim perintah melalui port serial, kemudian firmware memproses perintah tersebut dan mengeksekusi aksi pada aktuator, lalu mengirim balasan (reply) ke komputer sebagai indikator bahwa perintah telah diterima dan dijalankan.



Gambar 1. Kit iMCLab

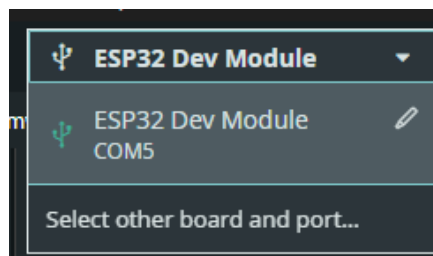
Perangkat iMCLab dihubungkan ke laptop menggunakan kabel USB sehingga port serial dapat terdeteksi dan digunakan untuk proses upload firmware maupun eksekusi pengujian yang ditunjukkan pada Gambar 1. Struktur folder proyek disusun agar seluruh file yang dibutuhkan berada dalam satu direktori, sehingga library `imclab.py` dapat diimpor langsung oleh skrip pengujian tanpa konfigurasi tambahan ditunjukkan pada Gambar 2. Dengan struktur ini, proses pengujian dapat dilakukan berulang kali dengan langkah yang konsisten, yaitu menjalankan skrip Python, mengamati respon pada terminal, dan memverifikasi perubahan kondisi LED serta motor pada kit iMCLab.



Gambar 2. Struktur folder proyek di komputer

3.2 Hasil Uji Koneksi dan Komunikasi Serial

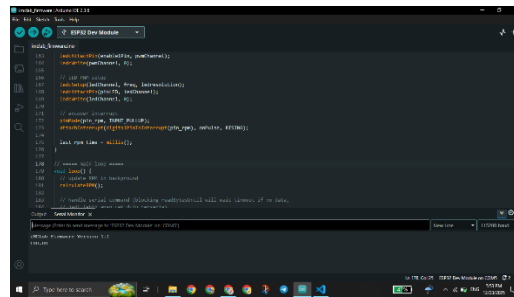
Pengujian koneksi dan komunikasi serial dilakukan untuk memastikan ESP32 pada kit iMCLab dapat berkomunikasi dengan komputer secara stabil sebelum menjalankan pengendalian LED dan motor. Tahap awal dimulai dari konfigurasi pada Arduino IDE dengan memilih board ESP32 Dev Module serta port COM yang terdeteksi pada sistem operasi ditunjukkan pada Gambar 3. Setelah konfigurasi benar, firmware berhasil diunggah ke ESP32, yang dibuktikan melalui log proses upload hingga selesai. Keberhasilan upload ini menjadi indikator bahwa perangkat terhubung dengan baik dan siap menerima perintah.



Gambar 3. Tampilan Arduino IDE: pemilihan board ESP32 Dev Module dan port COM.

Selanjutnya, validasi komunikasi dilakukan melalui Serial Monitor dengan baud rate 115200. Ketika perangkat di-reset, firmware menampilkan pesan `READY` yang menandakan program berjalan dan port serial aktif. Uji perintah `VER 0` juga menghasilkan balasan versi firmware, sehingga dapat disimpulkan bahwa mekanisme penerimaan perintah dan pengiriman respons sudah berfungsi. Pada tahap ini juga ditemukan bahwa port serial tidak dapat digunakan bersamaan oleh dua aplikasi; oleh karena itu Serial Monitor harus ditutup ketika pengujian dipindahkan ke Python agar koneksi tidak gagal. Dengan hasil tersebut, komunikasi serial dinyatakan berhasil dan sistem siap digunakan untuk pengujian pengendalian aktuator pada subbab berikutnya.

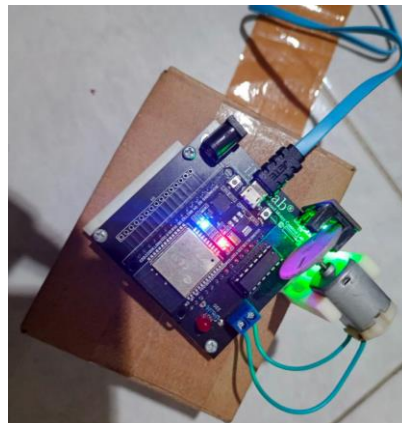
3.3 Hasil Uji Pengendalian LED



Gambar 4. Serial Monitor: uji LED 100 dan LED 0 + respons.

Pengendalian LED dilakukan untuk memverifikasi bahwa firmware mampu menerima perintah dari sisi host dan mengeksekusi keluaran PWM pada pin LED dengan benar. Skenario uji yang digunakan adalah memberikan nilai intensitas 0–100 melalui perintah LED, di mana nilai 0 merepresentasikan kondisi mati dan nilai 100 merepresentasikan kondisi menyala penuh. Pada pengujian melalui Serial Monitor, perintah LED 100 menghasilkan LED menyala dan firmware mengirim balasan nilai yang sama, sedangkan perintah LED 0 mematikan LED dan mengirim balasan nilai 0 ditunjukkan pada Gambar 4. Hasil ini menunjukkan bahwa parsing perintah, eksekusi PWM LED, dan mekanisme balasan firmware berjalan sesuai harapan.

Setelah validasi melalui Serial Monitor berhasil, pengujian dilanjutkan menggunakan skrip Python (*tes_led.py*) melalui library *imclab.py* untuk memastikan kontrol LED dapat dilakukan secara otomatis dari program. Pada sisi terminal, skrip menampilkan status koneksi hingga muncul pesan *READY*, kemudian mengeksekusi perintah LED ON dan LED OFF dengan jeda waktu tertentu. Selama pengujian, perubahan kondisi LED dapat diamati secara langsung pada kit iMCLab sesuai urutan perintah yang dijalankan pada gambar 5. Dengan demikian, pengendalian LED dinyatakan berhasil baik melalui antarmuka Serial Monitor maupun melalui pengujian berbasis Python, dan sistem siap digunakan untuk pengujian aktuator berikutnya yaitu motor DC.

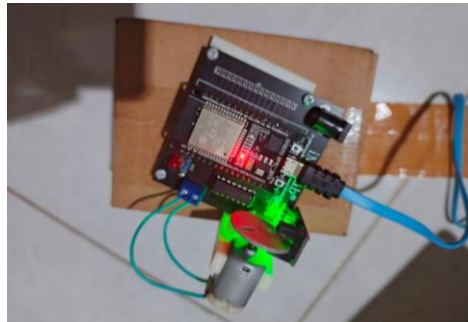


Gambar 5. LED berhasil menyala

3.4 Hasil Uji Pengendalian Motor DC

Pengujian pengendalian motor DC dilakukan setelah komunikasi serial dan kontrol LED berhasil, dengan tujuan memastikan perintah OP dapat mengatur keluaran PWM pada driver motor sehingga motor berputar sesuai nilai yang diberikan. Skenario uji dilakukan dengan mengirim nilai OP bertahap, misalnya 20%, 40%, 60%, 80%, hingga 100%, untuk mengamati perubahan kecepatan putaran secara kualitatif. Pengujian dijalankan melalui skrip Python dengan library *imclab.py*, dan terminal menampilkan nilai perintah serta balasan (*reply*) dari perangkat sebagai indikator bahwa perintah diterima ditunjukkan pada Gambar 6. Pada

nilai OP yang lebih tinggi, motor terlihat berputar lebih cepat dibandingkan nilai OP rendah, sehingga perubahan duty cycle PWM berpengaruh langsung terhadap kecepatan motor.



Gambar 6. Motor berhasil menyala

Selain uji bertahap, dilakukan uji putaran dalam durasi lebih lama pada nilai OP tertentu untuk memastikan motor dapat berjalan stabil selama waktu pengamatan. Selama pengujian, kondisi fisik motor dapat diamati secara langsung pada kit iMCLab, misalnya dari pergerakan poros/piringan yang terhubung dengan motor. Untuk menghentikan motor, perintah X digunakan sebagai fungsi stop, dan motor berhenti berputar setelah perintah tersebut dieksekusi. Berdasarkan hasil tersebut, dapat disimpulkan bahwa pengendalian motor DC melalui perintah OP dan X telah berjalan sesuai fungsi yang dirancang.

3.5 Pembahasan Kendala dan Perbaikan

Selama implementasi dan pengujian, beberapa kendala teknis ditemukan pada tahap upload maupun tahap eksekusi kontrol motor. Salah satu kendala yang muncul adalah kegagalan upload firmware pada ESP32 akibat perangkat belum masuk mode download, yang diselesaikan dengan melakukan prosedur tombol BOOT/EN saat proses koneksi sehingga proses flashing dapat berjalan hingga selesai. Kendala lainnya adalah kondisi Serial Monitor yang dapat “mengunci” port COM, sehingga skrip Python tidak dapat membuka koneksi ketika Serial Monitor masih aktif; solusi yang diterapkan adalah menutup Serial Monitor sebelum menjalankan pengujian berbasis Python.

Pada tahap pengendalian motor, kendala utama yang sempat terjadi adalah motor tidak berputar meskipun perintah OP sudah dikirim dan balasan muncul pada Serial Monitor. Setelah ditelusuri, penyebabnya berasal dari konfigurasi firmware, khususnya pemetaan pin PWM yang awalnya terpasang ke pin encoder/RPM, sehingga sinyal PWM tidak masuk ke pin enable motor. Perbaikan dilakukan dengan mengarahkan `ledcAttachPin()` ke pin enable motor dan menambahkan pengaturan pin arah motor sebagai output agar driver motor memperoleh sinyal kendali yang benar. Setelah perbaikan tersebut diterapkan dan firmware diunggah ulang, motor dapat berputar sesuai variasi nilai OP, dan fungsi stop X dapat menghentikan putaran dengan benar.

4. KESIMPULAN

Berdasarkan hasil implementasi dan pengujian, integrasi firmware ESP32 dan library Python berhasil diterapkan untuk pengendalian aktuator pada kit iMCLab melalui komunikasi serial USB. Perintah LED mampu mengatur kondisi nyala LED sesuai nilai yang diberikan dan memberikan balasan sebagai indikator eksekusi, sedangkan perintah OP mampu mengatur kecepatan putar motor DC secara kualitatif berdasarkan perubahan duty cycle PWM, serta perintah X dapat menghentikan motor dengan benar. Selain itu, penggunaan skrip Python

membuat proses pengujian lebih terstruktur dan repeatable karena skenario kontrol dapat dijalankan otomatis tanpa pengiriman perintah manual melalui Serial Monitor.

5. SARAN

Untuk pengembangan selanjutnya, sistem dapat ditingkatkan dengan menambahkan pembacaan sensor seperti encoder agar kecepatan motor dapat dipantau secara kuantitatif dan bukan hanya berdasarkan pengamatan. Kontrol motor juga dapat dikembangkan dari metode open-loop PWM menjadi closed-loop, misalnya menggunakan pengendali PID, sehingga kecepatan lebih stabil ketika terjadi perubahan beban. Selain itu, penambahan fitur pencatatan data (logging) dan visualisasi sederhana akan membantu proses analisis hasil pengujian serta memperkaya dokumentasi praktikum.

DAFTAR PUSTAKA

- [1] B. Rahmat, “iMCLab – Internet-Based Motor Control Lab,” GitHub Repository, diakses 26 Desember 2025.
- [2] Espressif Systems, “ESP32 Series Datasheet,” Espressif, diakses 26 Desember 2025.
- [3] Arduino, “Serial (UART) Reference,” Arduino Documentation, diakses 26 Desember 2025.
- [4] pySerial, “Welcome to pySerial’s documentation,” Read the Docs, diakses 26 Desember 2025.
- [5] pySerial, “Tools (including serial.tools.list_ports),” Read the Docs, diakses 26 Desember 2025.
- [6] Arduino, “Serial.readBytesUntil() Reference,” Arduino Documentation, diakses 26 Desember 2025.
- [7] Arduino, “Serial.println() Reference,” Arduino Documentation, diakses 26 Desember 2025.
- [8] Espressif Systems, “LED Control (LEDC) API — Arduino-ESP32,” Espressif Documentation, diakses 26 Desember 2025.
- [9] Random Nerd Tutorials, “ESP32 PWM with Arduino IDE (Analog Output),” diperbarui 11 Juni 2024, diakses 26 Desember 2025.
- [10] Arduino Forum, “Serial Input Basics (updated),” 25 April 2016, diakses 26 Desember 2025.
- [11] Espressif Systems, “LED Control (LEDC) — Arduino-ESP32 Documentation,” diakses 26 Desember 2025.
- [12] pySerial, “pySerial API,” diakses 26 Desember 2025.

- [13] pySerial, “Tools — serial.tools.list_ports,” diakses 26 Desember 2025.
 - [14] pySerial, “Short Introduction — Readline & Timeout,” diakses 26 Desember 2025.
 - [15] Espressif Systems, “esptool Troubleshooting (ESP32) — Wrong boot mode detected,” diakses 26 Desember 2025.
-